

Tic Tac Toe (C Program) – Line-by-Line Documentation

Purpose of This Document

This document explains the Tic Tac Toe program line by line. The goal is not just to describe what each line does, but to explain why it exists and how it affects the program flow. This is intended for learning and oral explanation.

Header File

```
#include <stdio.h>
```

Includes the Standard Input Output library. This allows the program to use printf() and scanf() for console input and output.

Function Prototypes

```
void printBoard();
```

Declares a function that prints the Tic Tac Toe board. The function is declared here so main() can call it before its actual definition appears in the file.

```
int checkWin();
```

Declares a function that checks whether the game is won, drawn, or still running. It returns an integer status code.

```
void system();
```

This declaration exists because the program calls system("cls") later. In proper C programs, system() should be declared by including <stdlib.h>. This declaration is not recommended but exists in this code.

Global Board Array

```
char board[] = {'0','1','2','3','4','5','6','7','8','9'};
```

This global array stores the game board. Index 0 is unused. Indices 1 to 9 represent the playable positions. Initially, each cell contains its number so empty positions are visible to the player.

Main Function

```
void main(){
```

The program starts execution here. Note that void main() is non-standard in C. The standard form is int main(void).

```
int player = 1;
```

Initializes the player counter. Player 1 always starts the game.

```
int input;
```

Stores the board position entered by the user (1 to 9).

```
int status = -1;
```

Represents the game state. -1 means the game is still running. This value controls the while loop.

```
printBoard();
```

Displays the initial empty board before any moves are made.

Game Loop

```
while (status == -1){
```

This loop keeps the game running until a win or draw occurs. In this program, the loop actually ends when main() returns after a win or draw.

```
player = (player % 2 == 0) ? 2 : 1;
```

Uses the modulo operator to decide which player's turn it is. Odd values correspond to Player 1, even values to Player 2.

```
char mark = (player == 1) ? 'X' : 'O';
```

Assigns the correct mark based on the current player.

```
printf("Please enter Number For Player %d\n", player);
```

Prompts the current player to enter a move.

```
scanf("%d", &input);
```

Reads the position number entered by the player and stores it in input.

```
if(input < 1 || input > 9){ printf("invalid input"); }
```

Checks if the input is outside the valid range. The program prints an error message but does not stop execution, which is a known flaw.

```
board[input] = mark;
```

Places the player's mark into the chosen board position. The program does not check if the cell is already occupied, which can lead to overwriting moves.

```
printBoard();
```

Prints the updated board after the move.

```
int result = checkWin();
```

Calls the function that determines whether the game has been won, drawn, or should continue.

```
if(result == 1){ printf("Player %d Wins", player); return; }
```

If checkWin() returns 1, the current player has won and the program terminates.

```
else if(result == 0){ printf("draw"); return; }
```

If checkWin() returns 0, the game is a draw and the program terminates.

```
player++;
```

Increments the player counter so the next turn switches to the other player.

```
}
```

Ends the while loop.

```
}
```

Ends the main function.

printBoard() Function

```
system("cls");
```

Clears the console screen. This works on Windows systems but is not portable.

The following printf statements format and display the board using board[1] through board[9]. Each row and column is printed to visually resemble a Tic Tac Toe grid.

checkWin() Function

This function checks all possible winning combinations (rows, columns, diagonals). If any three positions contain the same character, the function returns 1.

If no win is found, the function counts how many positions contain 'X' or 'O'. If all 9 positions are filled, the function returns 0, indicating a draw.

If neither a win nor a draw is detected, the function returns -1, meaning the game should continue.

Key Learning Points

- This program demonstrates game loops, conditional logic, arrays, and functions in C.
- It shows how state is managed using integer return values.
- It also highlights common beginner mistakes such as weak input validation and non-standard main().